DevOps — Zero to Master

23 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 What is DevOps?

02 Linux Basics

03 Networking Fundamentals

04 Shell Scripting

05 Git and Version Control

06 Web Servers

Intermediate (6 topics)

07 CI/CD Concepts and Tools

08 Docker Fundamentals

09 Dockerfile Best Practices

10 Docker Compose

11 Cloud Basics (AWS/Azure/GCP)

12 Package and Artifact Management

Advanced (7 topics)

13 Kubernetes Fundamentals

14 Helm and Kustomize

15 Infrastructure as Code: Terraform

16 Configuration Management: Ansible

17 Monitoring and Logging

18 Security and DevSecOps

19 GitOps

Master (4 topics)

20	SRE: SLIs, SLOs and Incident Response
21	Release Strategies
22	Scalability, Cost and Reliability
23	Real Project: End-to-end Pipeline

Beginner Level

01. What is DevOps?

DEFINITION

DevOps unites development and operations to ship software faster and more reliably through automation, collaboration and continuous feedback (CI/CD, IaC, monitoring).

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A restaurant where cooks and waiters work as one team with quick feedback instead of blaming each other; DevOps joins development and operations.

தமிழ்

சமையல்காரர்களும் சர்வர்களும் ஒருவரை ஒருவர் குறை சொல்லாமல் விரைவான கருத்துடன் ஒரு குழுவாக வேலை செய்யும் உணவகம் போல DevOps development, operations-ஐ இணைக்கிறது.

PROS

- Faster releases
- Fewer failures
- Better collaboration
- Automation reduces toil

CONS

- Needs culture change
- Tool sprawl
- Skills gap

ADVANCED / NEW FEATURES

- Platform engineering
- DORA metrics
- AIOps and AI-assisted operations
- Internal developer platforms

02. Linux Basics

DEFINITION

Navigate and manage systems from the command line: files, permissions, processes, packages, services.

SYNTAX

```
ls -la
chmod 644 f
systemctl status nginx
```

EXAMPLE

```
ls -la
chmod 644 file
ps aux | grep nginx
systemctl status nginx
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Knowing how to move around the godown, lock rooms and switch machines on; Linux commands manage servers.

தமிழ்

கிடங்கில் நடமாடுவது, அறைகளைப் பூட்டுவது, இயந்திரங்களை இயக்குவது போல Linux commands servers-ஐ நிர்வகிக்கின்றன.

PROS

- Powerful and free
- Runs most servers and containers

CONS

- Command line learning curve

ADVANCED / NEW FEATURES

- systemd and cgroups v2
- eBPF-based tooling

03. Networking Fundamentals

DEFINITION

IP, DNS, ports, HTTP/HTTPS, TCP vs UDP, load balancers, firewalls, SSH.

SYNTAX

```
ping host
curl -I url
ss -tulpn
```

EXAMPLE

```
curl -I https://example.com
ss -tulpn
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Understanding roads, house numbers (IP), and traffic signals (ports) for delivery; networking basics let services find and talk to each other.

தமிழ்

விநியோகத்துக்கு சாலைகள், வீட்டு எண்கள் (IP), சிக்னல்கள் (ports) புரிந்துகொள்வது போல networking அடிப்படைகள் சேவைகள் ஒன்றையொன்று கண்டுபிடித்துப் பேச உதவுகின்றன.

PROS

- Essential for debugging
- Applies everywhere

CONS

- Wide topic

ADVANCED / NEW FEATURES

- HTTP/3 and QUIC
- IPv6 adoption
- Service mesh

04. Shell Scripting

DEFINITION

Automate tasks with Bash: variables, loops, conditions, functions, exit codes.

SYNTAX

```
#!/usr/bin/env bash
set -euo pipefail
```

EXAMPLE

```
#!/usr/bin/env bash
set -euo pipefail
for f in *.log; do gzip "$f"; done
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A written checklist the helper follows every morning to open the shop; shell scripts automate repeated tasks.

தமிழ்

உதவியாளர் தினமும் காலை கடையைத் திறக்க பின்பற்றும் எழுதிய பட்டியல் போல shell scripts திரும்பத் திரும்ப வரும் வேலைகளை தானியக்கமாக்குகின்றன.

PROS

- Quick automation
- Available everywhere

CONS

- Error handling is weak
- Hard to maintain when big

ADVANCED / NEW FEATURES

- ShellCheck linting
- Python/Go for larger automation

05. Git and Version Control

DEFINITION

Branching, pull requests and trunk-based development are the base of automation.

SYNTAX

```
git switch -c feature/x
```

EXAMPLE

```
git switch -c feature/x
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

The shared notebook that records every change by every team member; Git is the base for automation.

தமிழ்

குழுவின் ஒவ்வொரு உறுப்பினரின் ஒவ்வொரு மாற்றத்தையும் பதிவு செய்யும் பகிரப்பட்ட நோட்டு போல Git automation-இன் அடிப்படை.

PROS

- Single source of truth
- Enables CI/CD and GitOps

CONS

- Branching discipline needed

ADVANCED / NEW FEATURES

- Trunk-based development
- Merge queues

06. Web Servers

DEFINITION

Nginx/Apache as web servers and reverse proxies; TLS certificates with Let's Encrypt.

SYNTAX

```
server { location / { proxy_pass http://app; } }
```

EXAMPLE

```
server { listen 80; location / { proxy_pass http://127.0.0.1:3000; } }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A receptionist who receives visitors and directs them to the right room; Nginx forwards requests to apps.

தமிழ்

பார்வையாளர்களை வரவேற்று சரியான அறைக்கு அனுப்பும் வரவேற்பாளர் போல Nginx requests-ஐ செயலிகளுக்கு அனுப்புகிறது.

PROS

- High performance
- TLS, caching, load balancing

CONS

- Config syntax needs care

ADVANCED / NEW FEATURES

- HTTP/3 support
- Caddy and Traefik alternatives
- Let's Encrypt automation

Intermediate Level

07. CI/CD Concepts and Tools

DEFINITION

Continuous Integration builds and tests every change; Continuous Delivery/Deployment releases automatically. Tools: GitHub Actions, GitLab CI, Jenkins.

SYNTAX

```
on: push
jobs:
  build:
    runs-on: ubuntu-latest
```

EXAMPLE

```
on: push
jobs:
  build:
    runs-on: ubuntu-latest
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

An assembly line that tests and packs every item as soon as it is made; CI/CD automates build, test and release.

தமிழ்

ஒவ்வொரு பொருளையும் உருவானவுடன் சோதித்து பேக் செய்யும் அசெம்பிளி வரிசை போல CI/CD build, test, release-ஐ தானியக்கமாக்குகிறது.

PROS

- Fast feedback
- Consistent releases

CONS

- Pipeline maintenance
- Flaky tests slow teams

ADVANCED / NEW FEATURES

- GitHub Actions/GitLab CI dominance
- Pipeline as code
- OIDC deployments

08. Docker Fundamentals

DEFINITION

Containers package apps and dependencies. Images, containers, volumes, networks, Dockerfile.

SYNTAX

```
docker run -p 8080:80 nginx
```

EXAMPLE

```
docker build -t myapp .
docker run -d -p 8080:80 myapp
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Standard tiffin boxes that carry the same food anywhere; containers carry an app and its dependencies anywhere.

தமிழ்

எங்கும் ஒரே உணவை எடுத்துச் செல்லும் தரப்படுத்தப்பட்ட டிபன் பெட்டிகள் போல containers செயலியையும் சார்புகளையும் எங்கும் கொண்டு செல்கின்றன.

PROS

- Consistent environments
- Lightweight vs VMs
- Fast start

CONS

- Security hardening needed
- Networking/storage complexity

ADVANCED / NEW FEATURES

- Docker Desktop and Docker Scout
- BuildKit
- Rootless containers
- Podman alternative

09. Dockerfile Best Practices

DEFINITION

Small base images, multi-stage builds, layer caching, non-root users, `.dockerignore`.

SYNTAX

```
FROM node:22-alpine AS build
...
```

```
FROM nginx:alpine
```

EXAMPLE

```
FROM node:22-alpine AS build
WORKDIR /app
COPY . .
RUN npm ci && npm run build
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Cooking in a big kitchen but serving in a small box; multi-stage builds keep final images small and secure.

தமிழ்

பெரிய சமையலறையில் சமைத்து சிறிய பெட்டியில் பரிமாறுவது போல multi-stage builds இறுதி images-ஐ சிறியதாகவும் பாதுகாப்பாகவும் வைக்கின்றன.

PROS

- Smaller, safer images
- Faster deploys

CONS

- Needs layer caching knowledge

ADVANCED / NEW FEATURES

- BuildKit cache mounts
- Distroless/Chainguard images
- SBOM and provenance

10. Docker Compose

DEFINITION

Define multi-container apps (app + database + cache) in YAML.

SYNTAX

```
services:
  web:
    build: .
  db:
    image: postgres:16
```

EXAMPLE

```
services:
  web:
    build: .
    ports: ["3000:3000"]
```

```
  db:
    image: postgres:16
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A thali plate with several items served together; Compose runs app, database and cache with one command.

தமிழ்

பல உணவுகள் ஒன்றாகப் பரிமாறப்படும் தட்டு போல Compose செயலி, database, cache-ஐ ஒரே கட்டளையில் இயக்குகிறது.

PROS

- Simple local environments
- Single file

CONS

- Not for large production clusters

ADVANCED / NEW FEATURES

- `docker compose watch`
- Compose Spec
- Profiles

11. Cloud Basics (AWS/Azure/GCP)

DEFINITION

Compute, storage, networking, IAM, managed databases; shared responsibility model.

SYNTAX

```
aws s3 ls
```

EXAMPLE

```
aws s3 ls
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Renting a godown, trucks and electricity as needed instead of buying them; cloud provides computing on demand.

தமிழ்

கிடங்கு, லாரி, மின்சாரத்தை வாங்காமல் தேவைக்கேற்ப வாடகைக்கு எடுப்பது போல cloud கணினி வளங்களை தேவைக்கேற்ப தருகிறது.

PROS

- Pay as you go
- Scale quickly
- Global reach

CONS

- Costs can spiral
- Vendor lock-in
- Security misconfiguration risk

ADVANCED / NEW FEATURES

- Serverless containers
- Multi-cloud tooling
- FinOps
- Sovereign cloud options

12. Package and Artifact Management

DEFINITION

Registries for images and packages; versioning, tagging and immutability.

SYNTAX

```
docker tag app:1.0 registry/app:1.0
```

EXAMPLE

```
docker tag myapp:1.0 registry.example.com/myapp:1.0
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A labelled store room for finished goods with version numbers; registries store images and packages.

தமிழ்

பதிப்பு எண்களுடன் முடிந்த பொருட்களுக்கான லேபிள் இடப்பட்ட கிடங்கு போல registries images, packages-ஐச் சேமிக்கின்றன.

PROS

- Versioned, immutable artifacts
- Easy rollback

CONS

- Storage cost
- Cleanup policies

ADVANCED / NEW FEATURES

- OCI artifacts
- Image signing with cosign
- SBOM attestations

Advanced Level

13. Kubernetes Fundamentals

DEFINITION

Pods, Deployments, Services, ConfigMaps, Secrets, Ingress; declarative manifests managed with `kubectl`.

SYNTAX

```
kubectl apply -f deploy.yaml
kubectl get pods
```

EXAMPLE

```
kubectl apply -f deployment.yaml
kubectl get pods
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A supervisor who keeps the right number of workers on duty, replaces absent ones and balances the load; Kubernetes orchestrates containers.

தமிழ்

சரியான எண்ணிக்கை தொழிலாளர்களை பணியில் வைத்து, வராதவர்களை மாற்றி, சுமையைச் சமப்படுத்தும் மேற்பார்வையாளர் போல Kubernetes containers-ஐ நிர்வகிக்கிறது.

PROS

- Self-healing
- Auto-scaling
- Huge ecosystem

CONS

- Complex to learn and run
- Resource overhead

ADVANCED / NEW FEATURES

- Sidecar containers GA
- In-place pod resize
- Gateway API replacing Ingress
- Managed K8s (EKS/AKS/GKE)

14. Helm and Kustomize

DEFINITION

Package and customise Kubernetes manifests per environment.

SYNTAX

```
helm install app ./chart -f values.yaml
```

EXAMPLE

```
helm install myapp ./chart -f values-prod.yaml
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A recipe with adjustable quantities for different occasions; Helm and Kustomize adapt manifests per environment.

தமிழ்

வெவ்வேறு நிகழ்ச்சிகளுக்கு அளவை மாற்றக்கூடிய செய்முறை போல Helm, Kustomize manifests-ஐ சூழலுக்கேற்ப மாற்றுகின்றன.

PROS

- Reusable packaging
- Environment overrides

CONS

- Template complexity

ADVANCED / NEW FEATURES

- Helm 4
- OCI chart registries
- Kustomize in kubectl

15. Infrastructure as Code: Terraform

DEFINITION

Provision cloud infrastructure with declarative code; state, modules, plan/apply.

SYNTAX

```
terraform init
terraform plan
terraform apply
```

EXAMPLE

```
terraform init
terraform plan
terraform apply
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

An architect's blueprint from which identical buildings can be built any time; Terraform builds cloud infrastructure from code.

தமிழ்

எப்போது வேண்டுமானாலும் ஒரே மாதிரிக் கட்டிடங்களைக் கட்டக்கூடிய கட்டிடக் கலைஞரின் வரைபடம் போல Terraform code-இலிருந்து cloud அமைப்பை உருவாக்குகிறது.

PROS

- Repeatable infrastructure
- Multi-cloud
- Plan before apply

CONS

- State file management
- Learning HCL

ADVANCED / NEW FEATURES

- OpenTofu open-source fork
- Terraform Stacks
- Policy as code (OPA, Sentinel)

16. Configuration Management: Ansible

DEFINITION

Agentless automation with playbooks and roles.

SYNTAX

```
- hosts: web
  tasks:
    - apt: name=nginx
```

EXAMPLE

```
- hosts: web
  tasks:
    - apt: name=nginx state=present
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A supervisor visiting many shops with the same checklist to set them up identically; Ansible configures many servers the same way.

தமிழ்

பல கடைகளுக்கும் ஒரே பட்டியலுடன் சென்று ஒரே மாதிரி அமைக்கும் மேற்பார்வையாளர் போல Ansible பல servers-ஐ ஒரே மாதிரி configure செய்கிறது.

PROS

- Agentless
- Simple YAML
- Idempotent

CONS

- Slower at large scale
- YAML logic gets messy

ADVANCED / NEW FEATURES

- Event-Driven Ansible
- Collections
- ansible-lint

17. Monitoring and Logging

DEFINITION

Metrics with Prometheus/Grafana, logs with ELK/Loki, alerts and dashboards; the golden signals.

SYNTAX

```
rate(http_requests_total[5m])
```

EXAMPLE

```
rate(http_requests_total[5m])
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

CCTV, a speed meter and an alarm in a shop to know instantly when something is wrong; monitoring and logging provide that.

தமிழ்

ஏதாவது தவறானால் உடனே தெரிந்துகொள்ள கடையில் CCTV, வேகமானி, அலாரம் போல monitoring, logging அதைத் தருகின்றன.

PROS

- Faster incident detection
- Data-driven decisions

CONS

- Alert fatigue
- Storage costs

ADVANCED / NEW FEATURES

- OpenTelemetry standard
- Grafana Loki/Tempo/Mimir
- eBPF observability

18. Security and DevSecOps

DEFINITION

Shift-left security: SAST/DAST, dependency and image scanning, secrets management (Vault), least privilege.

SYNTAX

```
trivy image app:1.0
```

EXAMPLE

```
trivy image myapp:1.0
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Security guards who check every parcel and every worker before work starts; DevSecOps builds security into the pipeline.

தமிழ்

வேலை தொடங்கும் முன் ஒவ்வொரு பார்சலையும் ஒவ்வொரு தொழிலாளரையும் சோதிக்கும் காவலர்கள் போல DevSecOps பாதுகாப்பை pipeline-இல் சேர்க்கிறது.

PROS

- Early vulnerability detection
- Compliance

CONS

- False positives

- Slows pipelines if misused

ADVANCED / NEW FEATURES

- SBOM and SLSA provenance
- Sigstore/cosign
- Secrets scanning
- AI-assisted triage

19. GitOps

DEFINITION

Git as the single source of truth with Argo CD or Flux reconciling clusters.

SYNTAX

```
argocd app sync app
```

EXAMPLE

```
argocd app sync myapp
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

The shop register in Git is the single truth, and a robot keeps the shelves exactly matching it; GitOps does that for clusters.

தமிழ்

Git பதிவேடு ஒரே உண்மை, ரோபோ அலமாரிகளை அதற்கு இணையாக வைக்கிறது; GitOps clusters-க்கு அதைச் செய்கிறது.

PROS

- Auditable
- Easy rollback
- Self-healing

CONS

- Learning curve
- Secrets handling needs care

ADVANCED / NEW FEATURES

- Argo CD and Flux
- Progressive delivery
- OCI sources

Master Level

20. SRE: SLIs, SLOs and Incident Response

DEFINITION

Define reliability targets, error budgets, on-call, runbooks and blameless postmortems.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop that promises 99.9% open hours and learns from every closure without blaming anyone; SRE sets SLOs, error budgets and blameless postmortems.

தமிழ்

99.9% திறந்த நேரம் உறுதியளித்து ஒவ்வொரு மூடலிலும் யாரையும் குறை சொல்லாமல் கற்றுக்கொள்ளும் கடை போல SRE SLO, error budget, blameless postmortems அமைக்கிறது.

PROS

- Reliability with balance
- Clear targets

CONS

- Cultural change
- Needs good measurement

ADVANCED / NEW FEATURES

- SLO-based alerting
- Incident automation
- Chaos engineering

21. Release Strategies

DEFINITION

Blue-green, canary and rolling deployments, feature flags, rollbacks.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Opening a new counter for a few customers first before opening it for everyone; canary, blue-green and rolling releases reduce risk.

தமிழ்

முதலில் சில வாடிக்கையாளர்களுக்கு மட்டும் புதிய கவுண்டரைத் திறந்து பின் அனைவருக்கும் திறப்பது போல canary, blue-green, rolling releases ஆபத்தைக் குறைக்கின்றன.

PROS

- Safer releases
- Quick rollback

CONS

- More infrastructure
- Needs good monitoring

ADVANCED / NEW FEATURES

- Argo Rollouts
- Feature flags
- Progressive delivery

22. Scalability, Cost and Reliability

DEFINITION

Autoscaling, caching, CDN, multi-region, backups and disaster recovery, FinOps.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Adding counters when the crowd comes and removing them when it goes, keeping backups ready; scalability, cost control and DR keep the business running.

தமிழ்

கூட்டம் வரும்போது கவுண்டர்களைச் சேர்த்து போகும்போது குறைத்து, காப்புப் பிரதிகளை தயாராக வைப்பது போல scalability, cost control, disaster recovery வணிகத்தைத் தொடர வைக்கின்றன.

PROS

- Resilience
- Cost savings with autoscaling

CONS

- Complex design
- Testing DR is rarely done

ADVANCED / NEW FEATURES

- FinOps practices
- Karpenter/KEDA autoscaling
- Multi-region active-active

23. Real Project: End-to-end Pipeline

DEFINITION

Code -> CI tests -> Docker image -> push to registry -> deploy to Kubernetes via Terraform + Argo CD, with monitoring and alerts.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A full pipeline from code to production with monitoring, like a complete food chain from farm to table.

தமிழ்

பண்ணை முதல் மேசை வரை முழு உணவுச் சங்கிலி போல code முதல் production வரை monitoring உடன் முழு pipeline.

PROS

- End-to-end skill practice

CONS

- Takes time to build

ADVANCED / NEW FEATURES

- Use GitHub Actions + Terraform/OpenTofu + Argo CD + OpenTelemetry

Interview Question Bank

Q1. What is DevOps?
A culture and set of practices combining development and operations to deliver software quickly and reliably.

Q2. CI vs CD?
CI merges and tests code frequently; CD automatically delivers (or deploys) validated builds.

Q3. Docker vs virtual machine?
Containers share the host kernel and are lightweight; VMs include a full guest OS.

Q4. What is Infrastructure as Code?
Managing infrastructure through versioned, declarative code (Terraform, CloudFormation).

Q5. What is Kubernetes?
A container orchestrator handling scheduling, scaling, networking and self-healing.

Q6. Blue-green vs canary deployment?
Blue-green switches traffic between two environments; canary releases to a small percentage first.

Q7. What is a reverse proxy?
A server that forwards client requests to backend servers (Nginx), adding TLS, caching and load balancing.

Q8. What is GitOps?
Declaring desired state in Git and having an operator reconcile the cluster to it.

Q9. What are the golden signals?
Latency, traffic, errors, saturation.

Q10. How do you manage secrets?
Secret managers (Vault, cloud KMS), environment injection, never in Git.

Capstone Project Ideas

1. Dockerise a full-stack app
2. CI/CD with GitHub Actions to a VPS
3. Terraform an AWS VPC + EC2
4. Kubernetes deployment with Helm
5. Prometheus + Grafana monitoring